Measuring Engineering Efficiency at LinkedIn

Learnings and insights from a principal engineer at LinkedIn and veteran of developer tools and productivity, Max Kanat-Alexander.

GERGELY OROSZ

NOV 15, 2022 • PAID

A few months ago, I tweeted an observation that so many developer productivity tools which vendors build, seem to be designed for CEOs and CFOs to buy them, not to make engineering teams more productive. This is a big difference! Max Kanat-Alexander, principal engineer at LinkedIn, replied, saying:

> "The more I read in this space, the more I want to publish what we are doing at LinkedIn."

Max is an expert when it comes to developer productivity; he's been in this segment for close to twenty years, spending the majority of it working on tools for software engineers. Over the past decade, he's spent more and more time working in the developer productivity problem space.

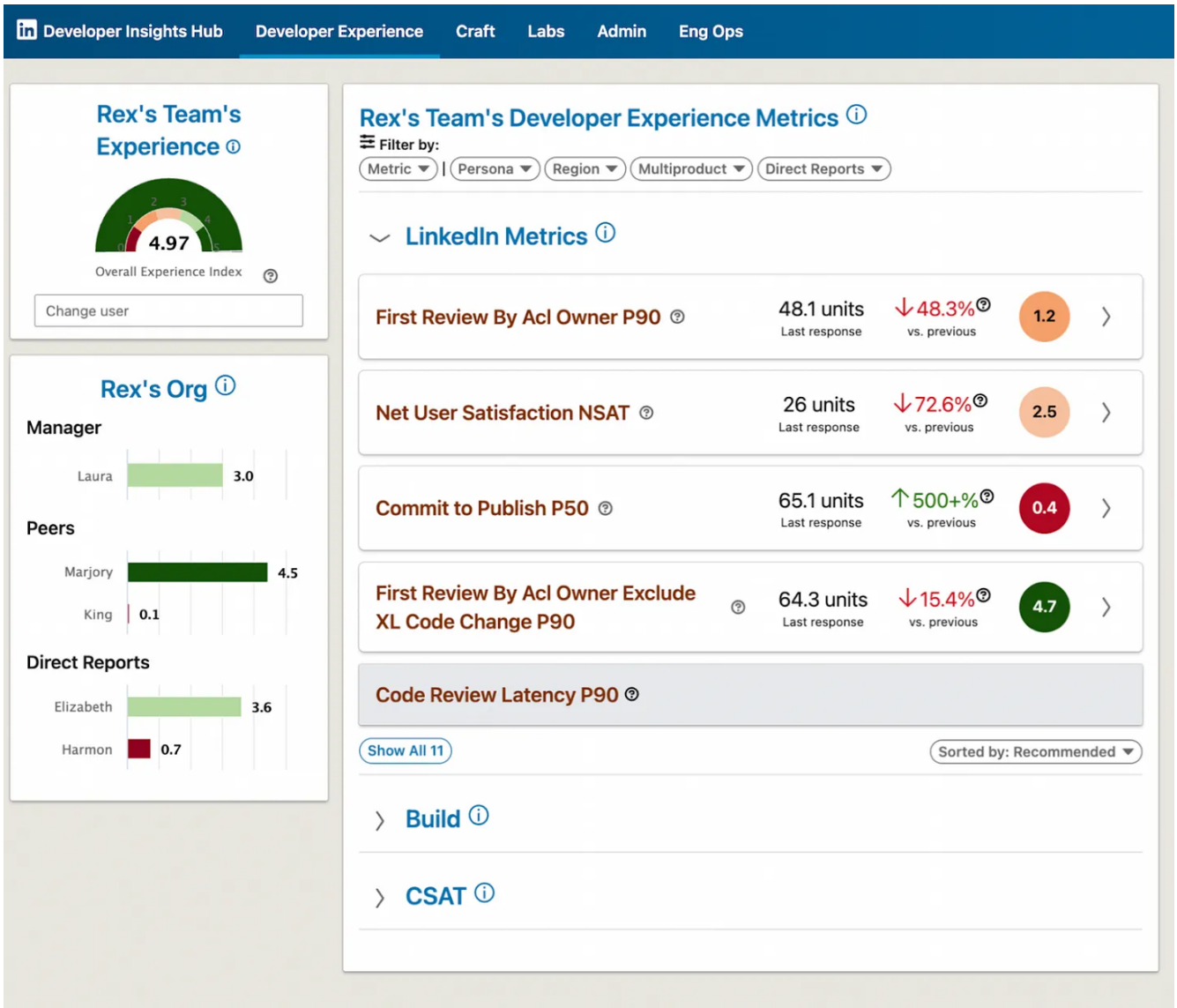
I jumped at the opportunity to learn more from Max about his experiences in this field, and to get a glimpse of what teams at LinkedIn are doing – and why.

In today's issue, we cover:

1. Max's career path
2. Developer experience learnings from Google
3. Developer productivity goals at LinkedIn
4. Developer productivity approaches at LinkedIn
5. Developer productivity dashboards

6. The road ahead
7. Why is developer productivity so hard to measure?

We'll also take a look at developer productivity dashboards that LinkedIn has built, like this one:



Screenshot of LinkedIn's Developer Insights Hub. All data is mocked-up.

With that, it's over to Max.

1. Max's career path: Bugzilla, Google, YouTube, LinkedIn

I worked on Bugzilla for eight years. I started my career as a technical support engineer at a company then-called Kerio, which made mail servers and firewalls. They wanted a bug tracker put in place, and the CEO asked if I had experience with Bugzilla. I'd done some volunteer bug triaging for the Mozilla project during college, so I was given the task to install Bugzilla for the company.

The problem was Bugzilla was *ugly*. However, there was a Red Hat fork that looked a bit nicer and which ran on Postgres, instead of MySQL which the main project used. I liked Postgres better, so I installed this version. However, Red Hat later abandoned this fork, which was a problem when we needed to upgrade. There I was, having to choose between migrating over their Postgres database to a MySQL one, or adding Postgres support to Bugzilla. Well, of course I decided to add Postgres support to Bugzilla! And I refactored the whole database layer while I was at it.

To get my changes out, I had to become the release manager of the Bugzilla project. To become a release manager, I had to start doing a lot of code reviews. And while doing all this, I worked my way into becoming one of the two co-leads of the Bugzilla project!

This is kind of how things go in open source. If you want to do the work, you will eventually be in charge of the thing. This might be less true on larger projects, but for small projects, this is how it goes. It's how I got into the Bugzilla project, and then did lots of refactoring, cleaning up of spaghetti code, and bringing the team along. Later, we became the most popular bug tracking system in the world!'

Eight years at Google. I joined Google, coming as the guy who was the architect of Bugzilla, a project written mostly in Perl and shipped in Linux boxes. On my first day at Google, my manager asked me, "Do you want to work on YouTube for the Xbox 360 in C# and Silverlight, developed on a Windows machine?" I said, "Okay, let me do it: it's a new and different experience."

And gosh, it was great. Visual Studio with C# was by far the best development environment I'd had up until that point. I don't think people realize how good Microsoft is at developer tools, and instead think of them as an enterprise company. But my view is that Microsoft is the best developer tools company in the world.

Within Google, I quickly gravitated towards developer productivity. I became the technical lead for Code Health across YouTube, helping developers become more productive, working on large-scale refactorings, development practices, test frameworks and education, across all YouTube teams. I did a year-long stint working on the Java platform, and then moved to be technical lead for Google's Code Health efforts. I was also the main author of Google's code review guidelines.

Three years at LinkedIn. I joined LinkedIn more than three years ago, to work on developer tools and productivity, which is where I am today.

2. Developer experience at Google

I worked for eight years at Google, a lot of this on developer productivity. Here are a few things I learned about this space at the company. Note that I'm no longer a Google employee, so my views don't represent that of Google.

At Google, I was a member of the Code Health group, and later led this collective. Within the group, we frequently had people show up to our meetings and ask:

"What metric should I use to measure code quality?"

We would tell them the same thing, every time:

"Don't use quantitative metrics to measure code *quality*."

People would come and ask us about capturing more exotic measurements, for example, "cyclomatic complexity," which measures the number of potential paths through a function. Over and again, we would explain these metrics are minimally useful, and that what matters for *code quality* are characteristics like simplicity, maintainability, and similar things.

Over time, I started to develop a deeper and more nuanced approach to the question of which metrics to measure:

1. "Simple code" means "easy to read, understand, and correctly modify." This was my first realization, as I built up a mental model of code quality.

2. Code quality is experienced by a human being, and isn't a quantity to be measured. Code quality is *not* a quantity. There's no "amount" of simple code. Instead, you have to find out from human beings what they are experiencing.

How to measure developer productivity? This was a question I kept getting during my tenure at Google. And it was not just from staff, but also from people outside the company. We knew commonly used metrics like "lines of code" were bad. A quote which I'm fond of about this, is from the computer scientist, Edsger W. Dijkstra, in 1988:

> "If we wish to count lines of code, we should not regard them as "lines produced" but as "lines spent": the current conventional wisdom is so foolish as to book that count on the wrong side of the ledger."

I had easily more than 100 conversations about this over the course of 10 years, with a variety of people. I eventually realized nobody could measure productivity because they had no idea what the word "productivity" meant.

When people can't figure out how to measure something, it's because they haven't defined the thing they're trying to measure. Measuring productivity was like trying to measure "foobar." No one I talked to could define what it was they wanted to measure.

In 2017, I ended up publishing some general thoughts on measuring developer productivity, in which I touch on the importance of defining what we want to measure.

I wrote a very popular internal document at Google about some suggestions for developer productivity measurement. It included a list of experiments we could do to progress in this field. In the end, I didn't get around to doing any of them.

Still, the popularity of the document was eye-opening. Developer productivity was something a *lot* of people cared about very deeply, and really wanted to solve!

Years after writing this internal document, I engaged in a project to reduce the time automated tests took to run. I worked on it for about three months and our work impacted tens of thousands of developers at Google, by optimizing lots of details on automated test execution.

As part of this project, I wanted to figure out if the project had accomplished anything that *mattered*. I dug through the results and confirmed we'd saved a lot of machine time, thanks to the tests running quicker. However, I could not find evidence that we'd saved human time, at least not in a way we were able to detect. I took my analysis a step further, and wrote a document in which I came to the conclusion that optimizing for human developer time is almost always the only worthwhile investment. I used actual numbers to come to this conclusion:

The value of human time at most software companies is 20-100x the value of machine time. This means it's possible to *lose money* by doing optimizations which save only machine time, but not any human time!

Towards the end of my tenure at Google, I started to read more documents and papers published by the company's Engineering Productivity Research team. I learned a lot from Ciera Jaspan, who was the Tech Lead of that group. *See Ciera's authored publications here*.

The "Goals, Signals, Metrics" framework is the most important thing I learned from Ciera, while at Google. This is a framework for defining metrics:

1. Define the actual goal you're trying to accomplish. Opt for a descriptive definition, for example, say "here is a description of what we want to actually accomplish with our work." Not the vaguer, "I want to measure X."
2. Define signals. How would you know if you accomplished that goal, if you had infinite knowledge of everything? These are called "signals." For example, a signal is "how happy are people with my product?" You cannot directly know how happy people are with your product, unless you're magically all-knowing. However, if having happy customers is one of your goals, then this is the right *signal* for that.
3. Figure out your metrics. These are proxies for that signal. Metrics are *always* proxies, there are no perfect metrics.

There's a lot more to know about that framework. In 2019, Ciera gave a talk where she walked through the framework and gave examples. It's been super useful.

One of the most effective things the Code Health group did at Google, was add a question on code complexity to the company's internal engineering survey. Adding a question to a Google-wide engineering survey is a big deal, because it makes leadership pay attention when areas don't get good scores. Most engineering managers want their teams to ship products, but also to make sure their reports are happy. If they get a survey result which indicates engineers are unhappy with an area, then most managers want to do something about it. They might not know what to do about it, but they at least become motivated to.

3. Developer productivity goals at LinkedIn

At LinkedIn, the project to measure developer productivity started when the leads of the Tools organization wanted to better understand the needs of our internal customers. To do that, we needed to look at our success with actual data from internal customers. The first meetings about understanding their needs through data took place just before I started at LinkedIn. Our original goals were to:

- Transform internal developer tools through a deep understanding of our developers' needs.
- Help transform engineering as a whole, through data-driven insights.

3.1 Transform engineering through data-driven insights

Across the software industry, many companies have adopted the "religion" of being data-driven when it comes to user-facing products. It's a big area of focus when training new Product Managers: how to collect data, how to present it, how to analyze it, and what to do with the results. However, this is not necessarily an area of focus for software engineers.

Most internal tooling teams don't have product managers. This means it's up to chance whether a team has the capability or know-how to plan feature roadmaps and priorities, based on data. When internal tooling teams do have PMs, they often struggle to have impact, meaning it's not a great fit for many. Still, not having a product manager creates a skill set gap.

Many internal tools teams without product managers do look at and operate on data. However, setting up data pipelines, creating effective and easy-to-understand visualizations, and putting a planning process in place which incorporates them, is not part of standard software engineer training, anywhere.

It's pretty much hit-or-miss as to whether an internal tooling team will operate on data insights. This is especially true when considering if they'd like to do so in a way that's deeply baked into the culture of the team.

When I say, "transform engineering as a whole through data-driven insights," *this* is what I'm talking about: to make it easy for *every* team to be able to operate on data about tools, productivity, craftsmanship, and related areas.

3.2 Setting realistic goals

How we went about setting 'realistic' goals is an interesting topic because there's different ways of being realistic.

1. **Conceptually realistic goals.** Is a goal possible to achieve as a concept, throwing tooling aside? For example, a goal that's not conceptually realistic is knowing the exact productivity of the company, at any second. This is not conceptually possible because "productivity" in that sentence doesn't mean anything. It's not realistic to measure something that doesn't exist or is undefined.
2. **Practical enough telemetry to measure goals.** What telemetry can you realistically get? How much investment will it take to be able to measure something, versus the value you'll get from measuring it? There are things you can measure with your current infrastructure. However, other things require either new data pipelines, or sometimes changes in the products themselves to record that data. There are times when you need to change product fundamentals, so you can have sensible measurements. For example, if you wanted to measure test statistics on a CI system, but that CI system was just a generic job orchestrator and recorded no data, then you'd have to add functionality to capture those data points.

You need to define your audience, together with your goals. Without specifying your audience, the goal will be a lot less practical. For example, take this goal, with an audience defined:

> "The SVP of Engineering wants to know if Working From Home caused it to take longer for code to get through code review."

For this, you have a "*who*" – the SVP of Engineering – and a semi-clear goal. You now have an answerable question.

As a note, I caution against answering such questions directly. Instead, create a system that can answer such questions; like a data pipeline, sensible tables, and a dashboard. Don't keep trying to answer one-off questions as you might get many more of them! Code review is definitely one of those areas for which you'd get lots of such questions!

If you leave out either the audience or the goal, then your results won't be sensible and could even be dangerous. For example, take the question:

"Let's measure the volume of changes being pushed to production."

Uh-oh. Are you going to let front-line managers measure individual ICs for this? That could lead to some managers starting to measure individual performance by the volume output of engineers. Such measurement can lead to an engineering culture which tries to push as many changes as possible, regardless of their quality or utility to the customer.

And there are other problems with this question. The type of dashboard you're going to build to answer this question is unclear. You're going to run into issues like, "do we count this particular type of automated behavior in the numbers we produce?" Depending on your audience, the answer will differ. This is why leaving out the audience is problematic.

Without specifying the goal, you get targets like:

"Give front-line managers a metric to measure test success rate."

Why? What will this metric accomplish for the business? What is likely to happen is an engineer will create a data pipeline and dashboard. Then, front-line managers might even use it. But there will likely be no impact on the business. The dashboard and data won't be set up to help you understand if you're accomplishing anything, or not. They'll just be numbers which people feel compelled to make better because of an executive mandate.

Throughout my career, I've seen teams behave irrationally in these circumstances. One team found a way to filter out some data from the data pipeline, which improved the metric. Then, this team sent out emails congratulating everybody for their great impact on the metric. But in reality, nothing had changed.

4. Developer productivity approach at LinkedIn

Here's the primary approach that we used to measure developer productivity.'

Start with specific goals, measurements and signals.

I wrote a document to get everyone on the same page with our goals. Here's a summary on what this document was about.

The simplest statement of our goal would be, "Developers at LinkedIn are **productive** and **happy**." That could use some clarification, though.

We can refine "developers at LinkedIn are productive," to "developers at LinkedIn are able to effectively and efficiently accomplish their intentions regarding LinkedIn's software systems."

And also, "happy" with what? Well, here's a more precise statement: "Developers at LinkedIn are happy with the tools, systems, processes, facilities, and activities involved in software development at LinkedIn."

A good part of this document clarifies those statements and goes into the signals we want to measure, so we know we're accomplishing those goals. We measure:

- Efficiency

- Effectiveness
- Happiness

These three signals applied nicely to every area for which we've made metrics, to date.

Collect the metrics to measure the signals. After the goals and signals were selected, a set of senior developers got together and collected all the metrics we thought we could make out of those signals. This turned out to be a 40-page document!

It gave us a broad overview of the space. With shared understanding, we could see what common infrastructure we might need to build across various areas. For example, we realized we would need a system to calculate "business hours" correctly, since many metrics only matter during business hours. For example, if you're measuring how long somebody is waiting for a code review response, you don't want to count weekends, holidays, or non-working hours.

Once we had all that, we had to decide what to actually work on.'

Developer cohorts, personas and persona owners

Instead of building a one-size-fits-all tooling for all software engineers, we did something different. **We segmented our developers into cohorts, based on their workflow.** We developed an understanding of the largest, distinct cohorts of developers at the company. These were the groups we identified:

- Backend
- Web
- iOS
- Android
- Data Scientists
- Machine Learning (ML) engineers
- SREs

Looking closer at the cohorts, the differences stood out. For example, our Backend developers wrote a lot of Java and mostly used IntelliJ. Our Web developers wrote mostly JavaScript or TypeScript and mostly used VS Code as their IDE.

We created "developer personas" for each cohort. We listed the tools they use for each phase of the development cycle. We also looked for a developer from each of these cohorts to be the representative of that cohort. We call them the "persona owner."

The persona owner wrote a summary document on the developer persona, and helped keep it up-to-date. They also ran with analyzing the feedback and surveys related to the cohort, presenting the data to infrastructure engineering leaders, and operating as the point-of-contact for engineers building the tools when they wanted more insights about the cohort.

This system — the Developer Personas and Persona Owners – is one of the most successful things we did for measuring engineering efficiency. With this model, we transformed the way infrastructure leaders think about internal users.

Without Developer Personas, it'd be all too tempting to take the largest group of developers and serve only them, typically backend engineers. However, smaller groups of developers are also key to the business. For example, the number of engineers doing Backend Java work is greater than the number of developers doing iOS work, but obviously our iOS platforms are very important for LinkedIn's members and customers. We need to think about the priorities of serving iOS engineers separately from how many of them we have. We need to look at the pain points of all Developer Personas and consider how we're going to prioritize our efforts, taking the business context into consideration as well.

Developer cohorts have been revolutionary at LinkedIn. This addition has been the most effective thing we put in place; it was the trigger which helped make so many other things happen.

A common problem when you work on infrastructure is that as a developer, you think you know what developers need. You're one of them, after all! For example, if you're a

Python developer, you might think that what developers need is to run their tests quickly, and the IDE needs to be simple and get out of the way.

However, if you're a Java developer, you'll probably think more along the lines of:

> "I need an intelligent IDE with rich features like automated refactoring, and an advanced test runner."

For me, it was eye-opening to go through this process of understanding how being productive meant very different things to different developers.

The universe of ML engineers is completely different from that of developers, for example. This was my most eye-opening experience. You're doing things with almost no relationship to anyone else in the software engineering field and the kind of pain you're going through, is unbelievable.

You might train a model for two days, meaning your iteration time is 2-3 days. You're likely writing Python code, then could run it for eight hours, only to discover there's a typo in your code or some other logic error that you didn't realize was there when you wrote it.

Compared with other engineers, ML engineers write minimal amounts of code and then deal with vast amounts of data explorations, feature selections and model analysis. They have huge pain points in testing these models and how to monitor them once in production.

Qualitative feedback

We instrumented all our developer tools so we could collect feedback in real time from developers, as they were performing a task. We do this in various ways:

1. **Rating and free-text feedback**. When a developer completes some workflow – for example, pushing a change through CI and into GitHub – we send them an email asking them to rate their experience and provide some free-text feedback. We had to instrument several systems to get this data, and then create sensible

dashboards so the information could be consumed by the right people at the right time.

2. **Developer productivity survey.** We run a developer productivity survey on a quarterly basis. It asks an overall question about satisfaction, and then questions about satisfaction with each tool. Since we have instrumented many systems for Real-Time Feedback, we also know if a developer has used a system during the quarterly period. So, we can customize our internal surveys to only ask people about tools they actually use.

You can't ask everybody about everything, all of the time. This seemingly simple and straightforward statement was one of our most important learnings. It seems obvious when you say it out loud, but implementing this insight is more complicated!

To implement the approach of asking developers different questions at different times, you use some math and observation. We needed to get statistically significant data over a certain period of time, for a certain population of developers. However, we set ourselves the constraint to only ask a developer one question, about one thing, every X number of days. So, we couldn't ask about 300 different tools at once; we needed to narrow our questions down to larger workflows. Even with this narrowing, we could not get enough data unless we asked everybody about all of them, all of the time.

Despite our efforts, we still needed to keep a developer survey in place. Developers are happier answering 30 questions in ten minutes, once a quarter, than getting a daily email with one question – even if it only takes 30 seconds to answer.'

Note from Gergely: For its own reasons, Amazon does get employees to answer one question per day via the Connections app, as covered in Inside Amazon's Engineering Culture.

Developers can specify how often they are willing to receive emails to answer questions about developer tools, outside of this quarterly survey. People can choose between once a week and once a month. Ultimately, we use both the real-time feedback system and the quarterly survey.

Quantitative feedback

We started off by building pipelines and dashboards for metrics which we thought would be the most approachable, already had some buy-in across the company, and which fitted into our Goals and Signals framework.

Our first dashboards were for Build Time, then we built basic Code Review metrics, and then we moved on to deployment success metrics and CI metrics.

Some had dashboards and pipelines, but were being maintained by a variety of engineers and teams across the company. Because dedicated engineers were not directly assigned to manage and maintain them, these dashboards were not considered reliable or informative, which resulted in very low use.

You need dedicated engineering resources to build and maintain engineering productivity dashboards. We put a dedicated team in place, decided what we intended to measure, set up reliable data pipelines, and made full-featured dashboards. Then, we actively maintained them all. The team which owns this area is a group of software engineers and data scientists.

Why did we need a dedicated team for all this? It can be a lot of work to build one of these pipelines, shake out the anomalies, make them reliable, and build dashboards which answer engineering managers' questions. Then there are constant changes to the systems we're measuring, or to the company's needs, that require us to modify the pipelines and dashboards. It's not a job you can give an engineer once for two weeks, then ignore forever afterwards. It's real engineering work which requires dedicated people.

Getting alignment on the definition of metrics is the lengthiest part of the process. This was a learning which surprised me; that it wasn't building dashboards which took the longest, but what happened before.

One thing which *really* helps get people onto the same page is having a "*decider*" in charge of writing down the metrics definitions themselves. I did this for many of our developer productivity metrics. I wrote a document explaining what we intended to measure, with enough specificity that an engineer could implement it.

This written definition of a metric doesn't contain details like, "use this data source and make this query." It says something like: "this is the definition of 'a deployment' and this is the definition of success. Here are the deployments we want to measure, and here is a basic description of an algorithm for how you add up these numbers to get a figure for deployment success."

As usual, when making telemetry changes and building the dashboards, it frequently turns out that something about the definition doesn't work. When this happens, the engineer talks with the decider, and they figure out how to proceed, and update the written definition of the metric.

Without a decider role, you quickly find yourself in a much longer decision-making process. This process can become so frustrating that you end up "designing by committee" and settling for a group of metrics that might not serve you very well.. The worst part of this process? It's that the process to design those metrics was *so* painful, that it's very hard to go back and change the definitions!

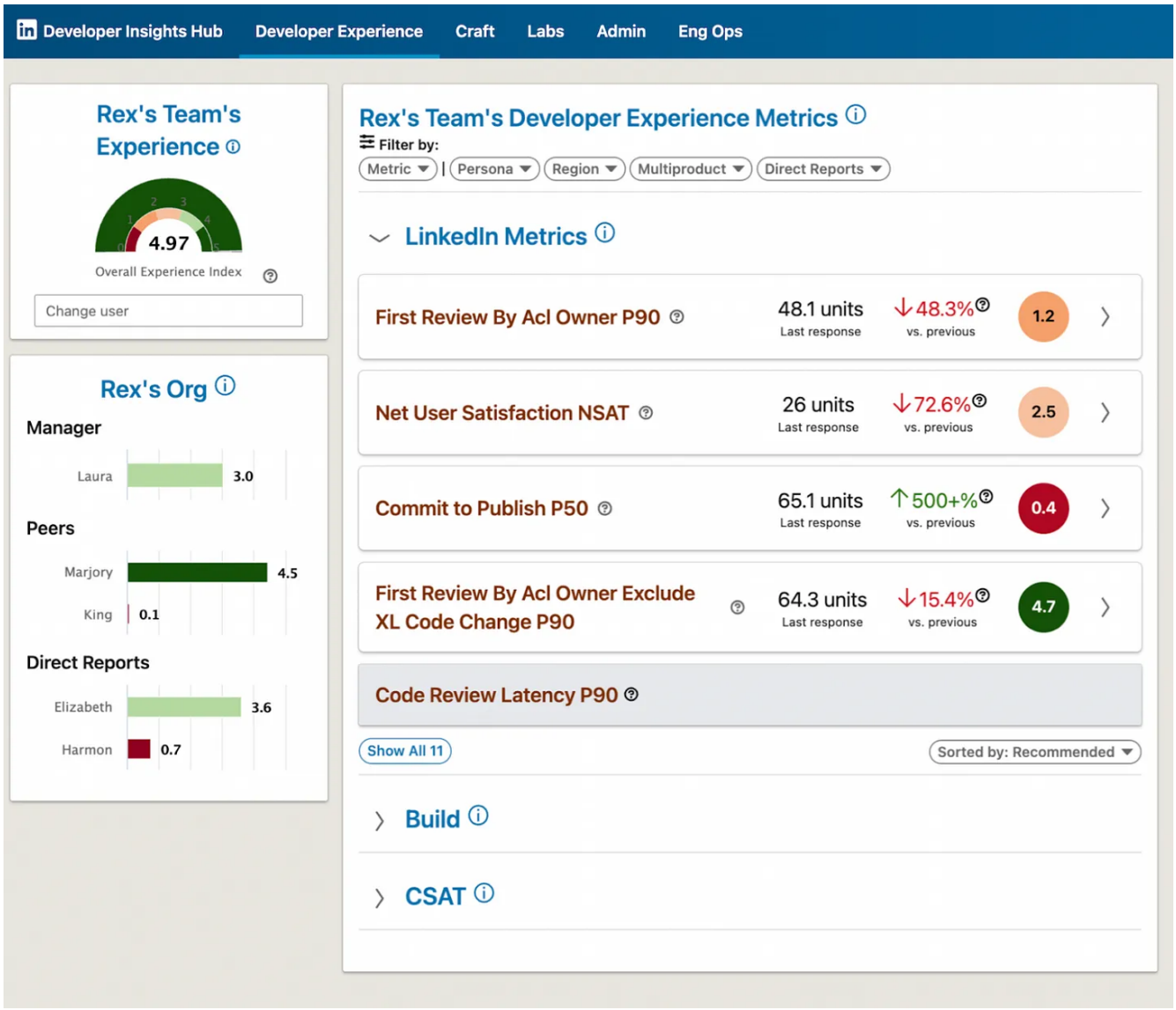
Even worse, when the engineer implementing these metrics runs into trouble, there's nobody whom they can ask, "what does the business intend to measure here?" All they can do is make an educated guess. However, all metrics have a lot of hidden assumptions baked into them. Years later, you'll discover some of these hidden assumptions, when you realize that the number you've been relying on doesn't actually measure what you thought it did!

Don't get me wrong: the decider should not be someone who sits in an "ivory tower" and never talks to anybody. Make sure the decider is somebody who has the social skills and connections to gather the information needed. And look, the decider will probably make incorrect decisions sometimes. And that's okay, at least with the decider method it's possible to change definitions as you go!

5. Developer productivity dashboards

Here's where we're at, today.

Developer Insights Hub: this is a dashboard for engineering leaders at LinkedIn. Here's what it looks like:



Screenshot of LinkedIn's Developer Insights Hub. All data is mocked-up.

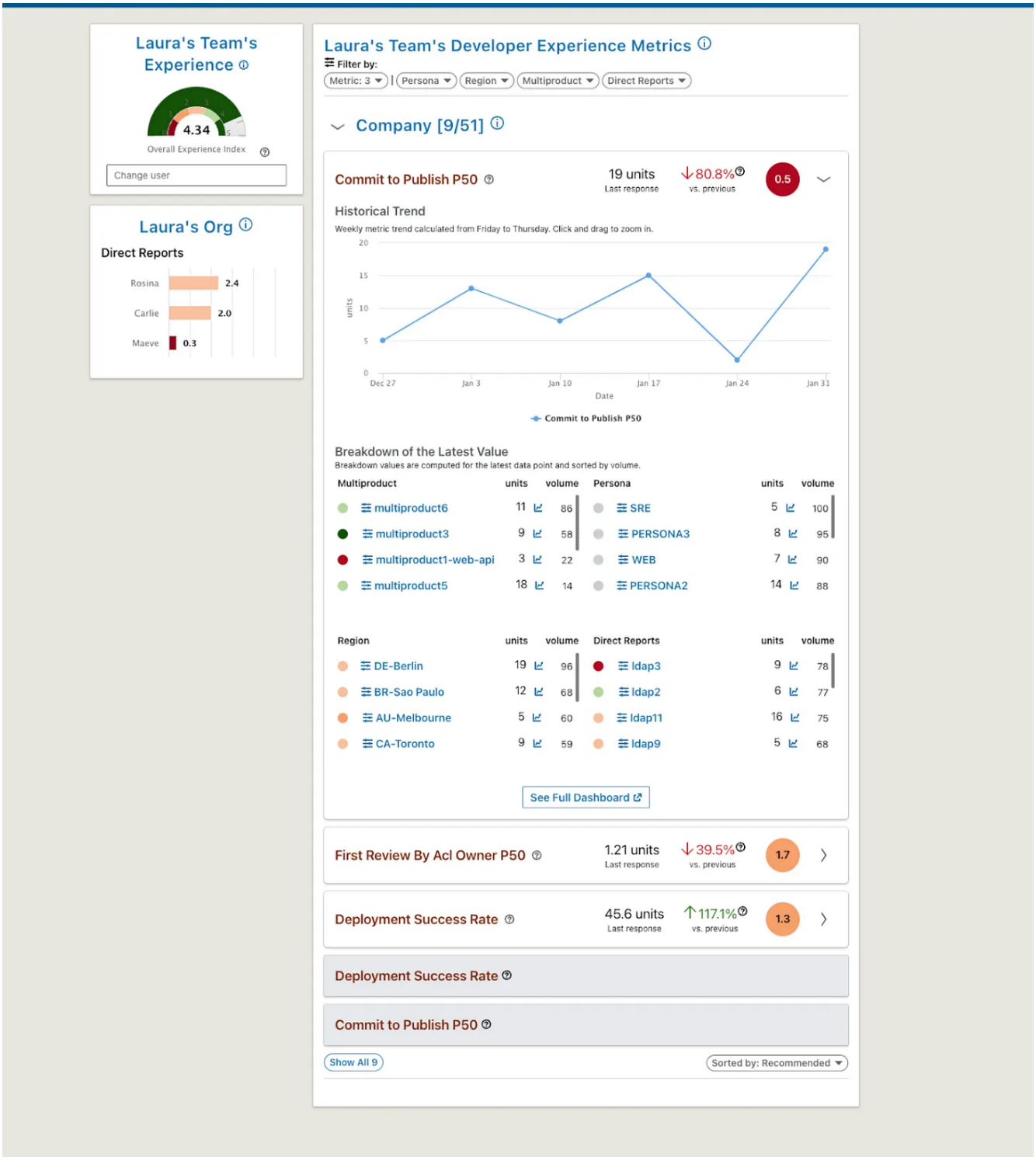
This is a system which lets managers get a quick, high-level overview of their organization's key developer productivity metrics. It provides trend graphs and a few key dimension filters for doing basic investigations of the data.

The general idea is this:

> "I'm a person who doesn't have a lot of time, and I don't want to dig into dashboards for hours. Show me the areas which need the most attention, so I can ask somebody

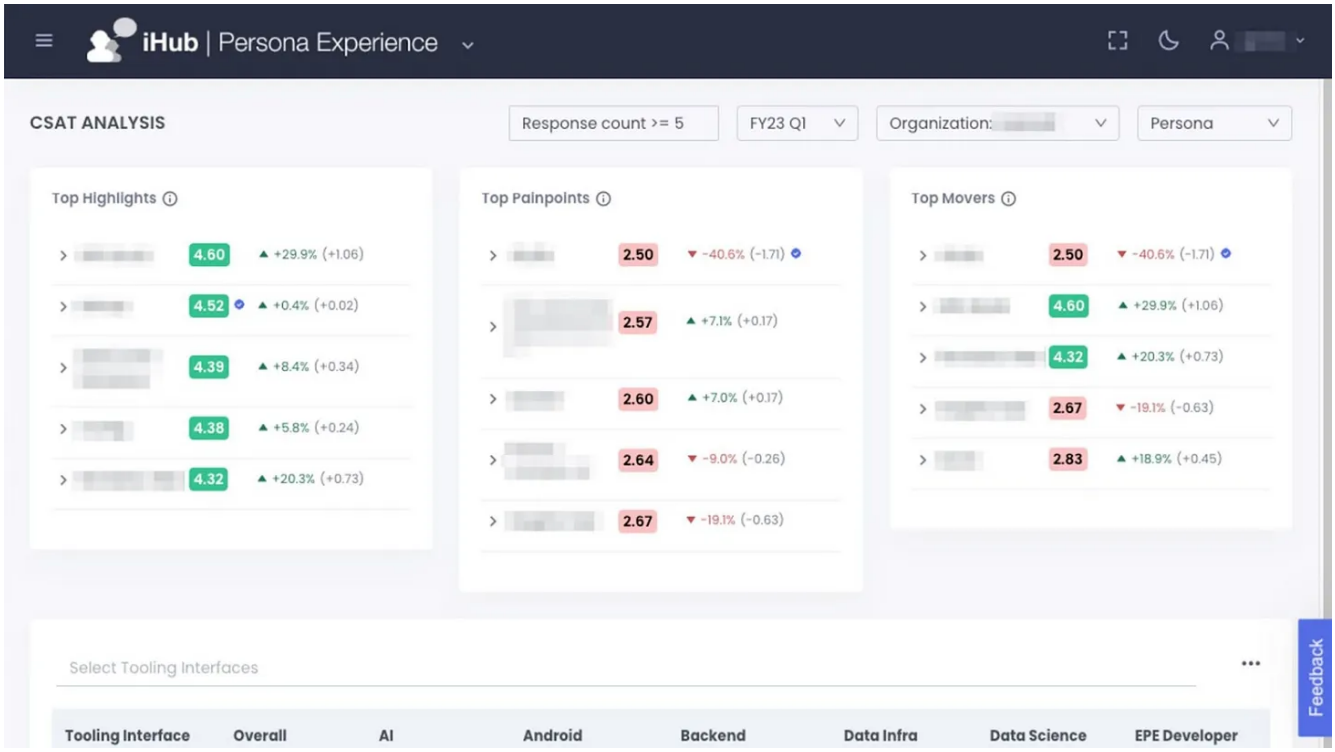
> to go look into them more deeply."

Each metric within the Developer Insights Hub links to a detailed dashboard, where an engineer or manager who wants to get into the details, can root-cause what's going on. Here's an example of drilling deeper into one of the several dimensions:



Drilling deeper into one of the metrics in the Developer Insights Hub; in this case, into Commit to Publish P50. All data is mocked-up.

The Persona Experience. Within the Developer Insights Hub – which we often refer to as iHub – we built a view for Persona Owners to get details about their cohort, aka the Developer Persona. Think of this as a survey analysis tool, but with a UI specialized for a particular set of users and dimensions relevant to them. We intend to expand this to be useful for all tool and infrastructure owners at LinkedIn. Here's how the tool looks like in production:



The Persona Experience view. Production data has been blurred out.

The feedback we've received so far about the dashboards has been interesting. Engineering leaders and Staff Engineers (or higher) have very positive feedback about our dashboard products. However, satisfaction scores are not good for ICs below Staff, which suggests they are not really being helped by the dashboards we've built.

This is actually what I expected, though. Dashboards are usually tools for managers or other engineering leaders, like Staff Engineers. I believe ICs "on the ground" are more helped by providing them with data *in their workflow*, which we want to do.

6. The road ahead

We've done a lot of work and we have so much more planned! Here are things we'd like to get to, eventually:

Broader developer productivity metrics. We are instrumenting the end-to-end developer workflow. This means putting instrumentation in place, starting from the code review phase, through to deployment. We do this to measure efficiencies at each step and compare those steps with each other, at a high level.

'When it comes to these metrics, our target audience is engineering leaders, and only high-level use cases. We do not plan to make it easy to measure anything and everything about any individual engineer's workflow.

Note from Gergely: as an interesting comparison, Uber decided to go exactly down the same route, of not allowing individual engineer's productivity to be measured, but only at the team level, as detailed in [How Uber is measuring developer productivity.]

Engineering flow and interruptions. I'd love to be able to see common sources of interruptions or difficulties across the company, so we can address them with tooling, process, or cultural changes.

Gathering all this data is very challenging because we don't own all the tools which could be interrupting somebody, such as email, calendar, Slack, etc. We also have to think very carefully about how we do this in a privacy-safe way. This is because you never want to create a system which can be used to "spy" on individuals. At the same time, done right, there are huge benefits to being able to properly understand what's *really* happening in workflows, without having to rely on anecdotal data from those who complain loudest.

Become a platform. Right now, our developer tools are products. However, these systems are becoming very popular internally. With their reach increasing, they'll have to stop being products and instead become platforms.

Becoming a platform will mean improvements like allowing people to onboard to them, without us – a central team – writing custom code, offering more ways for people to

build on top of them, and so on.

Use data to help developers during their work. For example, could we alert a developer that they are about to significantly regress their build time, before they make a commit? Could we surface this warning to them during code review, or even in their IDE, before they submit their code for review? Could we give people insights into the efficiency and effectiveness of their deployment plans, within the deployment tool itself?

On the ML side, we're tracking developments such as GitHub Copilot, and have been developing some internal projects which allow for similarly helpful functionality.

Expanding the developer productivity concept beyond software engineers. Many teams at LinkedIn work with infrastructure tools, and not all of them are built for software engineers. We build for sales associates, customer support representatives, recruiters, and other functions.

A question we'll want to answer is this: could we expand these developer platforms to help those tooling teams also measure success and to understand their customers' priorities?

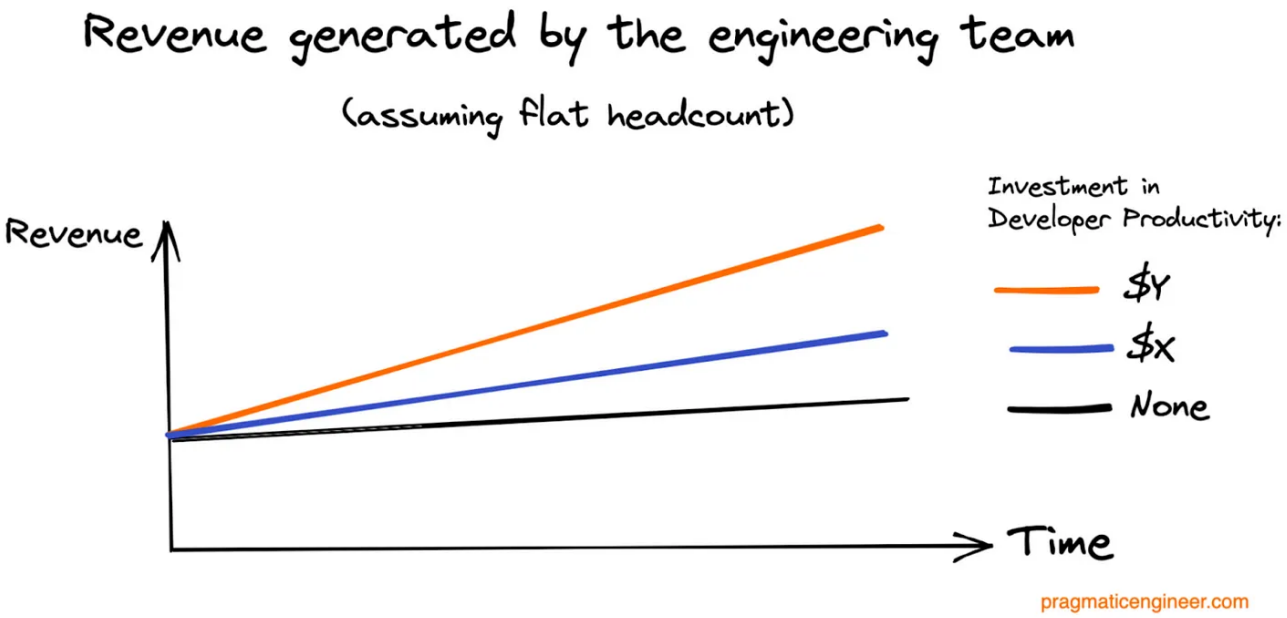
7. Why is developer productivity so hard to deal with?

Developer productivity seems like a slippery and hard-to-pin-down topic. Why is this?

People often overlook that we're dealing with humans, not machines. It's hit or miss as to whether a developer building developer efficiency tools will understand how developers *think*. Of course, a developer knows how they themselves think about development. But I've observed that many developers don't know how *other* developers think!

What causes distraction? Why are distractions bad? Which types of distractions are acceptable? What interruptions are acceptable? And how do you form a logical framework which predicts how things lead to good productivity?

It's hard to make an argument not for measuring productivity, but the things we can fix. Leadership often wants to hear how productivity will be increased by doing X, Y or Z and investing the amount it costs to achieve. Organizations love to see this kind of chart presented to them:



Investing in developer productivity: the presentation leadership would want to see to provide funding for initiatives.

However, the reality is you need to measure – and improve – those things within your control, which could have an impact on productivity.

The way promotions work also makes the business case for investing in developer productivity harder. At most companies, you have revenue sources which are more important for the company, and the engineering organizations close to these revenue sources are seen as profit centers.

For example, take a company which makes most of its money by selling ads. The Ads engineering team owns this problem space, but most other engineering teams contribute to the revenue that Ads generate. A good way to get promoted at such a company is to display how your project contributed to the Ads team, and how it – directly or indirectly – increased revenue from Ads.

However, teams which operate at the infrastructure or developer productivity level aren't always measuring clear impact metrics like that. This means it could become harder to make promotion cases for people working on these teams, and could also make funding additional headcount more challenging.

Takeaways

This is Gergely again.

Thanks so much to Max for sharing his journey and learnings, and to LinkedIn for signing off on sneak peeks of their developer productivity dashboards. Follow Max on Twitter and on LinkedIn. He is also the author of the books Code Simplicity (a free book) and Understanding Software.

Max has really got me thinking about *why* developer productivity is as elusive as it is. As software engineers, we like to think metrics including productivity, can surely be optimized?

However, during our conversation, Max identified several reasons why this isn't so:

- "Productivity" is meaningless without defining it in a much more specific way. Max opted for the "Goals, Metrics, Signals" framework, which takes a lot of the guessing out of what is meant by "productivity."
- In the real world, there's only so much you can measure. Even in software engineering, we can measure far less than we'd like to. For example, as Max mentioned, keeping track of all interruptions a developer gets is difficult to do in a way that respects privacy.
- Productivity is different for each developer. A Python engineer and an ML engineer have different workflows and "being very productive" has a different meaning for each.

Social skills are more important than technical skills, including when dealing with developer productivity! Max told me how one career learning has been that his technical skills matter far less than his social skills.

Even as we discussed the hardest aspects of measuring developer productivity, the factors raised were not the building of pipelines, or putting dashboards in place. Instead, the subject was getting people to agree on what to measure. Max put a 'decider' role in place, identified Persona Owners, and worked with these people – and also stakeholders – to keep everyone motivated and on the same page. The skill to do this does indeed have little to do with coding!

You cannot measure without the right observability metrics. So, build these! One thing that strikes me as different about how LinkedIn is approaching developer productivity, is that it is one of the few companies which doesn't just try to crunch Git or JIRA statistics, then call it a day.

Instead, the team at LinkedIn has worked backwards; asking what should be measured, then putting observability and telemetry in place to make this possible. This is much more work, and not something you can do with an off-the-shelf product. However, it has allowed Max and his team to get answers to the questions they care about, versus what a vendor would offer, based on ease of measurement.

Developer time is what matters, machine time is trivial. I don't think enough of us internalize just how trivial it is to improve machine time – such as build speed or test suites passing – compared to how difficult it is to improve how engineers use their time.

The example which Max shared has stuck with me. At Google, he spent months speeding up test runners, and this was seen as a success. At least, it was until Max investigated what improvements developers saw in their day-to-day work. And they reported that they saw none!

After more than a decade in this field, I sense the fire within Max to make further progress, is undimmed. During our conversation, I encountered a very curious but equally humble engineer, who is fascinated by how little we still know about what makes developers productive. I appreciate how open Max has been in sharing his reflections and plans, including how it will be interesting to see if developer productivity tooling is useful beyond software engineering.

To close, here's a thought from Max on whether DORA, SPACE or another developer productivity metric is better than others:

> "It's more important to define developer productivity metrics, than to pick some specific set of metrics and then follow them. Even if you choose the DORA metrics, or SPACE, or QUANTS (Google's internal framework) don't stick to them religiously. Become good at defining what you want to measure and who the audience is. You'll go further with this approach."

119 Likes

2 Comments

Write a comment...

Jeremy Hartley Writes Platform Product Management Nov 16, 2022

Thanks. This is an awesome article. There is a lot here to consume, but I will definitely looking further into the goals, signals, metrics framework and thinking more consistently about: "Developer time is what matters, machine time is trivial".

LIKE (5) REPLY •••

Michael Kubler Nov 20, 2022

+1 thanks for this article.

We've only got 5 developers (one is also a designer) and a product manager. That's the entire startup.

So we aren't at the point we need a dedicated team for this. But now I've got a lot better understand of how to think about developer productivity.

Thinking about different tools for different developer cohorts. Having people get asked questions about the tools and systems in place, etc.

LIKE (3) REPLY •••